Vox Scene-Selection Playbook

Vox Scene-Selection Playbook (Remotion)

A decision companion to `CATALOG.md`. The catalog answers *what scenes exist in the bench and whether they palette-lock*; this playbook answers *which scene type to reach for given the teaching job*, and points each one at the **verified bench keepers**. It consolidates two source passes (a learning-science scene matrix and an engineering/architecture pass), reconciles their conflicts, and flags every component name that is **not** in the downloaded bench. Grounded in Mayer's Cognitive Theory of Multimedia Learning (CTML).

The one rule everything else serves

Animation is not automatically better than a still. A meta-analysis of 26 studies found a medium advantage for instructional animation over static pictures — but only when the motion was **representational** (it shows change, sequence, or action), largest for procedural-motor content. When motion doesn't show change over time, direct attention, or chunk the narration, it is **decorative** — and decorative motion is extraneous cognitive load. This is the same discipline vox already enforces: media economy (most beats are own-graphics, stills are rare and placed at act boundaries) and "set up the problem and ask the question before answering it." Use Remotion for cleaner layout, timing, and progressive emphasis by default; escalate to heavier animation only when the concept is invisible motion or hidden structure.

CTML effects with the strongest evidence, and how they cash out here: **coherence** (cut decorative motion), **signaling** (cue the named element within ~5 frames), **spatial contiguity** (labels adjacent to their target, never a separate legend), **temporal contiguity** (visual enters as the narration names it), **segmenting** (chapter dividers, one active panel at a time), **pre-training** (definition cards before mechanism), **modality** (narration + minimal on-screen text, never full-paragraph duplication → the redundancy trap), **redundancy** (do not put the whole narration on screen).

Scene-type decision matrix

Vox fit = the catalog tag this scene type maps to. **Bench source** = verified keepers you actually have (see caveats section for what the source docs invented). **Reuse** = cross-domain reusability, 1–5.

Scene type	Use when	Learning purpose	Vox fit	Bench source (verified)	Avoid when	Reuse
Title card	Opening; resetting attention after a hard idea	Orientation, pre-training	TITLE	onda title-card; remotion-templates `cinematic-title-intro`, `title-split`; remotion-scenes Cinematic/Text	Viewer already knows the topic	5
Chapter / section	Switching subgoal or mechanism	Segmenting, signaling	TITLE	remotion-templates `chapter-title`; remotion-scenes Transition/Text; onda title	Video < 90s, or trivial switch	5

Scene type	Use when	Learning purpose	Vox fit	Bench source (verified)	Avoid when	Reuse
divider	inside one video			reuse		
Kinetic typography	The phrasing itself is the concept/contrast	Verbal emphasis, memorability	TEXT	remotion-scenes TextAnimations; onda WordStagger/Highlight/TrackingIn; remotion-bits AnimatedText; clippkit sliding/popping-text	Putting whole narration on screen	4
Word-by-word reveal	A term/quote needs paced decoding	Cueing, timed attention	TEXT	onda Captions/Typewriter; remotion-bits typewriter, blur-in, word-by-word; clippkit `typing-text`; remotion-ui TypeWriter	Long/math-dense sentences (redundancy)	4
Definition card	Introduce a term before using it	Pre-training, schema setup	TITLE/TEXT	onda callout/title; remotion-ui DataCard/ComparisonCard; clippkit card elements	Idea is process/tradeoff, not a term	5
Formula / equation reveal	Parse a symbolic relation step by step	Chunk symbols → meaning	SHAPE/TEXT	**@remotion/paths `evolvePath`** (first-party, add via npm); onda Underline/Highlight/Callout; KaTeX for the math	Derivation too long; symbols not pre-trained	4
Code walkthrough	Syntax / line-level reasoning matters	Procedural understanding	TEXT	remotion-bits `code-block`; Remotion `Sequence` for step timing; Shiki/Prism for tokens	A diagram would teach it faster	4
Data chart	A trend/comparison is the evidence	Quantitative reasoning	DATA	onda BarChart/LineChart/PieReveal/ProgressBar; remotion-templates area/comparison/donut/`chart-animation`; remotion-ui Bar/Line/PieChart; remotion-bits charts	One number is all that matters	5
Counter / statistic	A single number needs salience	Magnitude anchoring	DATA	onda CountUp/StatCard; remotion-templates `stat-counter`, `circular-progress`; remotion-ui MetricBlock/AnimatedNumber; remotion-bits animated-counter	A trend/baseline matters more	5
Timeline	Explanation is historical / causal-over-time	Temporal + causal order	SHAPE/DATA	onda Timeline; remotion-templates `progress-steps`; **@remotion/paths** + arrows	Order is logical but not temporal	4
Process / flow diagram	Multiple parts interact causally	Mental-model building	SHAPE	onda NodeGraph/Callout/BoundingBox; remotion-ui FlowDiagram; **@remotion/shapes** arrows	It's really a simple sequence	5
Step-by-step diagram	Learner follows an ordered procedure	Procedural chunking	SHAPE/TEXT	onda ProgressSteps; remotion-templates `progress-steps`; remotion-bits list/grid-stagger; Remotion Series/Sequence	Branching, loops, or concurrency	5
Comparison table	Structured attribute comparison	Relational discrimination	DATA	remotion-templates `comparison-chart`; remotion-ui ComparisonCard; **@remotion/layout-utils**	> 3–4 entities, or mobile-first	4
Before / after	A state change is the message	Contrastive understanding	SHAPE	remotion-templates `image-comparison-slider`, `split-screen`; onda SplitScreen	Intermediate states matter more	4
Map / geography	Location or spread matters	Spatial grounding	— (not in bench)	**None kept** — use @remotion/mapbox or MapLibre example + @remotion/paths	Geography is incidental	3
System architecture	Hidden structure/interfaces matter	Mental model of a system	SHAPE	onda NodeGraph/BoundingBox/Callout; remotion-ui flow; **@remotion/shapes**	You need runtime traces or code	4
Molecular / cellular / physics	The phenomenon is invisible + dynamic	Causal intuition, unseen process	EFFECT/3D	remotion-bits particle-system, scene-3d; remotion-scenes particles/effects; **@remotion/three**	A still diagram already carries it	3
2D abstract shape	Concept is relational, not photographic	Analogical explanation	SHAPE	remotion-scenes ShapeAnimations; remotion-templates geometric; **@remotion/shapes / paths**	Abstraction obscures the concept	4
3D concept	Depth / rotation / occlusion is central	Spatial reasoning	EFFECT/3D	remotion-bits scene-3d, 3D carousel; **@remotion/three**	A 2D schematic teaches it faster	3
Camera / pan / zoom	A dense visual needs guided inspection	Attention guidance, contiguity	EFFECT/CAMERA	remotion-templates `ken-burns`, `parallax-pan`; onda KenBurns/Parallax/Spotlight; remotion-bits ken-burns	Base visual already readable	5
Audio waveform / reactive	The audio itself is the content	Auditory attention, pacing	AUDIO	clippkit bar/circular/linear-waveform; onda AudioVisualizer; remotion-ui WaveformVisualizer; remotion-templates `sound-wave`	Waveform is decorative filler	3

Scene type	Use when	Learning purpose	Vox fit	Bench source (verified)	Avoid when	Reuse
Quiz / checkpoint	You want a pause for retrieval	Recall, self-check	TITLE + timing	remotion-templates `countdown-timer`, `progress-steps`; onda title + ProgressSteps (build the card from primitives)	Too fast to allow real retrieval	5
Summary / recap	Closing a section or the video	Consolidation, transfer	TITLE	onda EndCard/TitleCard; remotion-templates `end-card`; remotion-bits list-reveal; remotion-ui title/card	Introducing new ideas	5

Decision tree — pick by the learner's next cognitive task, not by what looks animated

Ask what the learner must *do* next: orient, define, compare, inspect, model change, localize, retrieve, or consolidate.

First, categorize the material. **Conceptual / structural** (relational architectures, transitions, spatial arrays) → structural branch. **Procedural / sequential** (logic branches, loops, steps, math operations) → procedural branch. **Data / numerical** (metrics, comparisons, statistics) → quantitative branch.

Structural branch: a spatial delta between two states → **before/after** with a sync'd slider; nested system nodes → **system architecture**, scaling nodes in sequence; molecular/particle coordinates → **molecular/physics** built on math models, with **camera pan/zoom** to shift focus; earth coordinates → **map/geography** on a flat projection.

Procedural branch: linear, no branching → **step-by-step** with a stable rail and one changing panel; branches/loops/recursion → **process/flow diagram**, drawing each connector only *after* its target node has entered; source code → **code walkthrough** with active-line dimming; symbolic transformation → **formula reveal**, one clause per beat.

Quantitative branch: a single number/constant → **counter/stat**; continuous multi-variable trend → **data chart**, drawing left-to-right with one highlighted series.

Orientation and closure wrap everything: **title card** to establish the question, **definition card** to pre-train the key noun, **chapter divider** at each subgoal, **quiz/checkpoint** to convert watching into retrieval, **summary/recap** to consolidate — never a new idea at the end.

The negative rule: if the motion doesn't show change, direct attention, or chunk the narration, it's decorative — keep the scene but drop the animation to layout + timing + progressive emphasis.

Recommended palette for a 5-minute science explainer

The reliable palette is not twenty scene forms — it's a small set repeated with disciplined pacing: **orient → pre-train → show mechanism → show evidence → check understanding → recap.** That maps to segmenting/signaling and to Remotion's strength as a repeated-scene system.

Time	Scene	Why it's in the palette
0:00–0:15	Title card	Establishes the question at low cognitive load
0:15–0:35	Definition card	Pre-trains the key noun before the process starts
0:35–0:55	Chapter divider	Signals the core mechanism, resets attention
0:55–1:55	Step-by-step diagram	Best default for "how it works" — chunks causality on a stable rail
1:55–2:35	Process / flow diagram	Expands from linear steps to interacting parts
2:35–2:55	Counter / stat	Magnitude anchor before the evidence details

Time	Scene	Why it's in the palette
2:55–3:35	Data chart	What the evidence says — one narrated takeaway at a time
3:35–4:05	Camera-zoom or before/after	Guided inspection of the densest visual
4:05–4:25	Quiz / checkpoint	Converts explanation into retrieval
4:25–5:00	Summary / recap	Three takeaways, optional end card, no new info

Strategic substitutions, keeping the palette small: invisible spatial structure → swap a middle mechanism scene for **molecular/physics or 3D**; code-centric → swap the flow diagram for a **code walkthrough**; geographic → swap the inspection scene for a **map**.

Ranked scene types for high-volume production

Reconciled from the two source passes (which disagreed — see caveats). Ranked by cross-domain transfer, narration-friendliness, cognitive-load safety, and actual bench support — not visual complexity.

1. **Step-by-step diagrams** — nearly every domain explains procedures; maps cleanly to Series/Sequence.
2. **Definition cards** — reduce later overload via pre-training; trivial to templatize.
3. **Data charts** — cross-domain evidence scene, strong bench support.
4. **Summary / recap** — high value, low cost, modular.
5. **Title cards** — universal orienting scene, low risk when brief.
6. **Chapter dividers** — cheap segmenting and structure.
7. **Process / flow diagrams** — externalize causal structure (science, medicine, AI, engineering).
8. **Counter / statistic** — reusable magnitude anchor, easy to parameterize.
9. **Camera / pan / zoom** — reusable over any dense diagram, still, or figure.
10. **Code walkthrough** — domain-specific but high value where syntax matters.
11. Comparison tables · 12. Before/after · 13. Timelines · 14. Formula reveals · 15. Kinetic typography · 16. Quiz/checkpoint · 17. Maps · 18. System architecture · 19. Word-by-word reveal · 20. Audio waveform · 21. 2D abstract shape · 22. Molecular/physics sims · 23. 3D concept.

Default to the top ten; reach for 3D, waveform-reactive motion, or highly abstract animation only when the objective plainly requires them.

Internal library taxonomy (how this slots into vox)

Structure the bench as families with shared motion, accessibility, and content contracts — three tiers by state complexity:

Atom tier — stateless layout + tokens: `FadeIn`, `SlideIn`, `Stagger`, plus `colors.ts` / `spring.ts`. One overdamped house spring (a `SPRING_SMOOTH`) enforces a consistent motion language. This is where the vox palette (cream/ink/teal/crimson) lives as tokens — the single place to retint the whole bench.

Molecule tier — single-concept blocks: `DefinitionCard`, `StatBlock`, `TextReveal`, `LatexFormula`, `Waveform`. Each exports a strict **Zod schema** beside its typed props — the validated data contract that lets a beat-sheet drive it safely. (onda and remotion-ui already ship this pattern; they're the cleanest to lift.)

Organism tier — state-driven systems: `FlowDiagram`, `BarChart` / `LineChart`, `CodeWalkthrough`. Fully timeline-driven — visual state is a pure function of `useCurrentFrame()`, no side effects, no mouse handlers.

Every scene should expose the same slots: `content`, `layoutPreset`, `motionPreset`, `durationInFrames`, `safeArea`, `a11yPreset`, `themeTokens`. Assets in `public/` via `staticFile()`; timing orchestrated with `Composition` / `Sequence` / `TransitionSeries`. **This is the same contract as the vox slot model** — a beat declares a shot, the compiler conforms it to `actual_duration_s` — so a Remotion scene fills a beat by rendering to `media/<BID>.mp4` and inheriting the beat's duration.

Reconciliation caveats — what to trust and what the source got wrong

Trust the scene→purpose→pattern logic and the cognitive-load rules; they're well-grounded and reinforce vox doctrine. Do **not** trust the component attributions without checking the catalog:

- **remocn is mostly gone.** The source repeatedly credits remocn for typography, code blocks (Glass Code Block / Terminal), transitions, and UI (Button/Select/Dialog). We kept **exactly one** remocn unit (`wave-wipe`); its 43 others were UI chrome / chat-flow marketing. Anywhere the source says "remocn," substitute onda / remotion-bits / remotion-templates.
- **Invented or absent components.** `IBM/chuk-motion`, `mafs-remotion` / `mafs-remotion-animation`, `remotion-globegl`, `remocn Ecosystem Constellation`, `template-three` — none are in the downloaded bench. Equation math needs KaTeX/MathJax + `@remotion/paths`; maps need `@remotion/mapbox`; 3D needs `@remotion/three` — all added via npm, not present as bench keepers.
- **No map or dedicated equation scene in the bench.** Those two rows are build-from-primitives, not lift-and-restyle.
- **Conflicting reuse scores.** The two passes disagree (kinetic typography 4/5 vs 2/5; timeline 4/5 vs 3/5; process/flow 5/5 vs 3/5). This playbook takes the learning-science pass's scores as primary; treat any single score as a hint, not a spec.
- **Some recommended scenes are cut categories.** Quiz/checkpoint is credited to remocn-ui and remotion-scenes UIAnimations — both cut as UI. Build the quiz card from title + progress primitives instead.
- **First-party is underused.** `@remotion/paths` (`evolvePath` draw-on, `warpPath` morph), `@remotion/shapes`, and `@remotion/layout-utils` are the most transferable tools for equations, diagrams, and tables, and the source barely leans on them. There is no first-party chart or equation package — build charts with SVG + `interpolate()`/D3.